PPGCC PROGRAMA DE PÓS-GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO INF INSTITUTO DE INFORMÁTICA UFG UNIVERSIDADE FEDERAL DE GOIÁS

Arquitetura, Integração e Viabilidade

Sistema Model-Driven com Grafos de Conhecimento e LLMs

Mestrando: Fernando Souza Furtado Carrilho

Orientador: Prof. Dr. Fábio Moreira Costa

Goiânia - GO · **2026**

Resumo Executivo

A proposta apresenta uma arquitetura neuro-simbólica baseada na integração de DSLs, Grafos de Conhecimento e restrição gramatical de LLMs, visando reduzir alucinações e garantir validade semântica. A solução utiliza componentes consolidados da literatura combinados com módulos próprios para garantir independência de modelo.

1. Validação da Arquitetura e Componentes de Terceiros

A arquitetura é tecnicamente válida e fundamentada na integração de Engenharia Dirigida a Modelos (MDE), Grafos de Conhecimento e Geração Restrita.

Componentes Selecionados

- **Neo4j (Banco de Grafos):** Atua como núcleo da camada semântica. É o padrão ouro para modelagem de grafos e navegação eficiente via Cypher.
- **Impromptu / Langium (MDPE):** Permite geração de prompts orientados a modelos e gramáticas, garantindo portabilidade.

- **Guidance / LM-Format-Enforcer:** Força a conformidade estrutural das respostas do LLM com gramáticas BNF.
- **LangChain:** Responsável pela orquestração do pipeline e integração entre grafo e LLM.

2. Componentes a Implementar

Objetivo

Para garantir independência de modelo (LLM-agnostic), os seguintes módulos devem ser desenvolvidos.

- **DSL Parser & Graph Mapper:** Compilador que converte a DSL em nós e relações no Neo4j.
- **Semantic Routing Engine:** Interpreta o prompt do usuário e ativa subgrafos relevantes via consultas Cypher.
- **Dynamic Grammar Builder:** Gera dinamicamente regras BNF a partir do subgrafo ativado.

3. Plataforma, Linguagens e Frameworks

A stack prioriza flexibilidade, integração com IA e escalabilidade.

- **Linguagem Principal:** Python 3.10+
- **Banco de Dados:** Neo4j (AuraDB ou local)
- **Orquestração:** LangChain
- **LLMs:** GPT-4o ou Llama-3
- **API:** FastAPI
- **Protocolo:** MCP Server (opcional)

4. Estratégia de Integração

Pipeline model-driven orientado a microsserviços.

1. DSL é definida e convertida em grafo no Neo4j.
2. Usuário envia prompt via API (FastAPI).
3. LangChain ativa o Semantic Routing Engine.
4. Subgrafo relevante é recuperado.
5. Grammar Builder gera regras BNF.
6. LLM gera saída estruturada (JSON).
7. Resultado é retornado ao sistema cliente.

5. Plano de Testes e Avaliação

A validação envolve aspectos formais (engenharia) e probabilísticos (IA).

Métricas e Abordagens

- **Testes Unitários:** DSL $\rightarrow$ Cypher (pytest)
- **Recuperação Semântica:** Context Precision (CP) e Context Recall (CR)
- **Validade Estrutural:** Pass@k e taxa de erro sintático $\rightarrow$ objetivo: 0%
- **Métricas Textuais:** BLEU-1 e ROUGE

6. Análise de Viabilidade

Viabilidade: Alta

A proposta reduz significativamente alucinações ao restringir semanticamente e estruturalmente a geração dos LLMs, eliminando dependência de fine-tuning custoso.

Mês	Fase	Atividades Principais
1	**Fundamentação**	Definição da DSL e configuração do Neo4j
2	**Pipeline Inicial**	Implementação do Parser e Graph Mapper
3	**Prompting**	Desenvolvimento do Grammar Builder + integração com LLM
4	**Integração**	Criação do motor semântico + LangChain + API
5	**Testes**	Avaliação com métricas (BLEU, ROUGE, Pass@k)
6	**Finalização**	Refatoração e documentação

7. Cronograma (6 Meses)

Documento técnico com foco em arquitetura neuro-simbólica e engenharia dirigida a modelos.

8. Justificativa da Escolha Tecnológica

A seleção das tecnologias foi orientada por critérios de desempenho em grafos, capacidade de integração com LLMs, maturidade do ecossistema e aderência à arquitetura model-driven proposta.

- **Python 3.10+ (Linguagem Principal):** Python foi escolhido devido ao seu amplo ecossistema em IA e PLN. Frameworks como LangChain e bibliotecas de grafos possuem suporte nativo. *Comparação:* Alternativas como Java ou C# oferecem maior performance em alguns cenários, porém possuem menor flexibilidade com LLMs.
- **Neo4j (Banco de Dados de Grafos):** Selecionado por ser um banco nativo de grafos com *index-free adjacency*. A linguagem Cypher é altamente expressiva para recuperação semântica. *Comparação:* ArangoDB oferece maior flexibilidade (multi-modelo), porém menor otimização para grafos puros.
- **LangChain (Orquestração):** Permite a construção de pipelines complexos envolvendo LLMs e ferramentas externas. *Comparação:* LlamaIndex é focado em RAG de documentos, enquanto LangChain oferece maior controle sobre fluxos e agentes.
- **LLMs (GPT-4o / Llama-3):** O uso de modelos proprietários e open-source garante flexibilidade. *Comparação:* Modelos menores possuem menor generalização em tarefas estruturadas.

- **FastAPI (Camada de API):** Alta performance assíncrona, tipagem forte e fácil integração. *Comparação:* Flask é menos performático em cenários concorrentes; Django não é otimizado para APIs leves.
- **MCP Server (Protocolo de Comunicação):** O *Model Context Protocol* expõe o grafo como ferramenta para agentes de IA, promovendo interoperabilidade. *Comparação:* APIs REST exigem maior acoplamento estático.

9. Criação da Linguagem Específica de Domínio (DSL)

Para assegurar que as restrições do negócio sejam respeitadas, é necessário construir uma *Interpreted Domain-Specific Modeling Language* (i-DSML). Diferente de abordagens que compilam modelos para código-fonte, uma i-DSML permite a execução direta do modelo, capturando o conhecimento explícito e validando as operações em tempo de execução.

9.1 Arquitetura Conceitual da DSL

A formulação exigirá a definição de quatro pilares essenciais:

- **Sintaxe Abstrata (Metamodelo):** Define as regras estruturais e de integridade da linguagem (ex: `Drone`, `Placa_Solar`).
- **Sintaxe Concreta:** Representação textual, estruturada obrigatoriamente através de uma gramática *Backus-Naur Form* (BNF).
- **Domínio Semântico:** O significado de cada construção lógica.
- **Mapeamentos:** A transição dos artefatos para o Grafo de Conhecimento.

9.2 Ferramentas e Frameworks Selecionados

A implementação desta camada fará uso de tecnologias de ponta voltadas para a Engenharia Dirigida a Modelos (MDE).

9.3 Passo-a-Passo: Especificação e Implementação da DSL

A seguir, detalhamos o fluxo operacional prático utilizando o *Langium*.

Etapa 1: Definição da Gramática (BNF) via Langium

A primeira etapa consiste em definir a gramática de forma declarativa. O formato BNF dita como as instruções devem ser escritas para que o *parser* consiga interpretá-las.

Ferramenta	Camada de Atuação	Justificativa e Propósito
Langium	Construção da DSL	Framework de código aberto focado em engenharia de linguagem. Gerará automaticamente a Árvore de Sintaxe Abstrata (AST).
GME / Xactium	Metamodelagem	Ferramentas baseadas em MOF para a definição visual do metamodelo e validação arquitetural.
Impromptu	Eng. de Prompts	DSL voltada para *Model-Driven Prompt Engineering* (MDPE). Compilará prompts independentes impondo regras ao LLM.

Exemplo Prático (Domínio Drone/Placa Solar):

DroneInspection.langium

```
grammar DroneInspection

// Regra inicial
entry Model: (diagnostics+=Diagnostic)*;

// Definicao de Entidades e Relacionamentos
Diagnostic:
    'analisar' placa=ID 'status' estado=Estado 'acao_exigida'
        acao=Acao;

// Tipos permitidos (Restricoes estruturais)
Estado returns string: 'Fumaca' | 'Alta_Temp' | 'Poeira' | '
    Limpo';
Acao returns string: 'Transmitir_5G' | 'Retornar_Base';
terminal ID: /[_a-zA-Z][\w_]*/;
```

Etapa 2: Geração da Árvore de Sintaxe Abstrata (AST)

Após a escrita do código BNF, o compilador do Langium gera a **AST**. A AST é uma representação orientada a objetos (em `TypeScript` ou `Python`) de tudo o que foi descrito. É neste ponto que a validação estrutural acontece. Se o drone tentar registrar uma ação inválida, o validador nativo disparará um erro sintático imediato, impedindo que essa informação chegue ao LLM.

Etapa 3: Mapeamento para o Grafo e Prompting

1. **Alimentando o Grafo:** Um *script* processa a AST validada e a converte em instâncias de controle no banco de dados (Neo4j).
2. **Integração com o Impromptu:** Com o contexto mapeado, utilizamos a DSL **Impromptu** para orquestrar atributos extraídos do Grafo, gerando *prompts* modulares e reutilizáveis.
3. **Interface com o LLM:** O prompt final gerado é enviado ao LLM, que atua unicamente como tradutor semântico encapsulado pelas barreiras formais.